

Лабораторная работа №13

дисциплина: Архитектура компьютера

Безлепкина Татьяна Игоревна

Содержание

Цель работы	5
Задание	6
Теоретическое введение	7
Выполнение лабораторной работы	8
Выводы	11
Контрольные вопросы	12
Список литературы	15

Список иллюстраций

Список таблиц

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

В ходе лабораторной работы требуется разработать четыре скрипта (и одну программу на C/C++):

- Командный файл с getopt: Скрипт анализирует ключи: `-inputfile` (входной файл), `-outputfile` (выходной файл), `-p` (шаблон поиска), `-C` (учёт регистра), `-n` (номера строк). На основе этих ключей с помощью `grep` выполняет поиск и выводит результат в файл или на экран.
- Программа на C/C++ и скрипт-анализатор: Программа вводит целое число и завершается с кодом `exit(1)`, `exit(2)` или `exit(0)` в зависимости от знака числа (плюс обработка ошибки ввода). Командный файл запускает эту программу, анализирует код возврата через `$?` и выводит сообщение: «число больше нуля», «меньше нуля» или «равно нулю».
- Создание и удаление нумерованных файлов: Скрипт принимает число `N` (аргумент командной строки) и создаёт `N` файлов с именами `1.tmp`, `2.tmp`, ..., `N.tmp`. При втором аргументе `delete` эти файлы удаляются (если существуют).
- Архивация изменённых файлов: Скрипт упаковывает в архив `tar` все файлы из заданной директории. В модифицированном варианте архивируются только файлы, изменённые менее недели назад (используется `find -mtime -7`).

Теоретическое введение

В операционной системе UNIX командный процессор (оболочка, shell) является не только средством для выполнения команд, но и мощным языком программирования. Shell-скрипты позволяют автоматизировать повторяющиеся задачи, управлять файлами и процессами, использовать условные операторы, циклы, функции и обрабатывать аргументы командной строки. В данной лабораторной работе особое внимание уделяется: Разбору ключей командной строки с помощью встроенной команды `getopts`, которая позволяет гибко обрабатывать опции и их аргументы. - Ветвлениям – конструкциям `if-then-else`, `case` и проверке кодов возврата программ (переменная `$?`). - Циклам – `for`, `while`, `until`, их синтаксису и отличиям. - Генерации имён файлов с помощью метасимволов (`*`, `?`, `[]`), которые подставляются оболочкой. - Взаимодействию с программами на языках C/C++ через коды завершения `exit()`. - Архивации файлов командами `tar` и `find`, в том числе с фильтрацией по времени изменения. - Эти знания необходимы для написания эффективных и надёжных командных файлов в среде UNIX.

Выполнение лабораторной работы

На рисунке 1 представлен листинг командного файла search.sh. В нём используются конструкции getopt для разбора ключей -i, -o, -p, -C, -n, а также условный оператор if для проверки существования входного файла и перенаправление вывода грер. Скрипт реализует поиск строк по шаблону с

```
GNU nano 8.5
# Переменные
inputfile=""
outfile=""
pattern=""
case_sensitive="" # Будет "-i" при ключе -C, иначе пусто
line_numbers=""

# Разбор ключей
while getopts "i:o:p:C:n" opt; do
    case $opt in
        i) inputfile="$OPTARG" ;;
        o) outfile="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="-i" ;; # -C включает игнорирование регистра
        n) line_numbers="-n" ;;
        *) echo "Неверный ключ. Используйте: $0 -i файл -p шаблон [-o выходной файл]"
           exit 1 ;;
    esac
done

# Проверка обязательных аргументов
if [ -z "$inputfile" ] || [ -z "$pattern" ]; then
    echo "Ошибка: нужно указать -i и -p"
    exit 1
fi

# Проверка существования входного файла
if [ ! -f "$inputfile" ]; then
    echo "Ошибка: файл '$inputfile' не найден"
    exit 1
fi
```

возможностью учёта регистра и вывода номеров строк (рис. ??)

На рисунке 2 показан процесс выполнения скрипта. Командой chmod +x search.sh заданы права на выполнение. Затем скрипт запущен с ключами: -i test.txt -p "Hi" -C -n -o result.txt. В результате в файл result.txt выведены строки, содержащие "Hi" без учёта регистра, с номерами строк (рис. ??)

```
tibezlepkin@fedora:~$ chmod +x search.sh
tibezlepkin@fedora:~$ ./search.sh -i test.txt -p "Hi" -C -n -o result.txt
Ошибка: файл 'test.txt' не найден
tibezlepkin@fedora:~$ touch test.txt
tibezlepkin@fedora:~$ nano test.txt
tibezlepkin@fedora:~$ ./search.sh -i test.txt -p "Hi" -C -n -o result.txt
Результат записан в result.txt
```

Рисунок 3 — листинг checksum.cpp. Программа вводит число и (рис. ??)


```

GNU nano 8.5                                checksum..
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
    int num;
    cout << "Введите целое число: ";

    if (!(cin >> num)) {
        cerr << "Ошибка ввода" << endl;
        exit(255);
    }

    if (num > 0)
        exit(1);
    else if (num < 0)
        exit(2);
    else
        exit(0);
}

```

Рисунок 4 — листинг run_check.sh. Скрипт запускает checksum, анализирует \$?

```

GNU nano 8.5                                run_check.sh
#!/bin/bash

# Вывод программы
./checksum

# Анализ кода возврата ($?)
case $? in
    0) echo "Число равно нулю" ;;
    1) echo "Число больше нуля" ;;
    2) echo "Число меньше нуля" ;;
    *) echo "Ошибка выполнения (код $?)" ;;
esac

```

через case и выводит сообщение (рис. ??)

Рисунок 5 — выполнение. Компиляция g++ checksum.cpp -o checksum, запуск run_check.sh. При вводе 5 — «больше нуля», -2 — «меньше нуля», 0 — «равно

```

tibezelepkins@fedora:~$ g++ checksum.cpp -o checksum
tibezelepkins@fedora:~$ chmod +x run_check.sh
tibezelepkins@fedora:~$ ./run_check.sh
Введите целое число: 9
Число больше нуля

```

нулю», abc — «ошибка». (рис. ??)

Рисунок 6 — листинг files.sh. Скрипт создаёт файлы 1.tmp ... N.tmp через цикл

```

GNU nano 8.5                                file.sh
#!/bin/bash

# Проверяем, передан ли аргумент
if [ $# -lt 1 ]; then
    echo "Ошибка: нужно указать количество файлов"
    echo "Использование: $0 N [delete]"
    echo "N - количество файлов (1.tmp, 2.tmp, ...)"
    echo "delete - удалить созданные файлы"
    exit 1
fi

# Сохраняем количество файлов
N=$1

# Проверяем, что N - это число
if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "Ошибка: N должно быть положительным числом"
    exit 1
fi

# Проверяем, что N больше 0
if [ "$N" -eq 0 ]; then
    echo "Ошибка: N должно быть больше 0"
    exit 1
fi

```

for, если второй аргумент delete — удаляет. (рис. ??)

Рисунок 7 — выполнение. ./files.sh 5 — созданы пять файлов, ls проверяет.

```
tibezlepkina@fedora:~$ nano file.sh
tibezlepkina@fedora:~$ chmod +x file.sh
tibezlepkina@fedora:~$ ./file.sh 5
Создаю 5 файлов...
Создан 1.tmp
Создан 2.tmp
Создан 3.tmp
Создан 4.tmp
Создан 5.tmp
Готово. Создано 5 файлов.
tibezlepkina@fedora:~$ ./file.sh 5 delete
Удаляю файлы...
Удалён 1.tmp
Удалён 2.tmp
Удалён 3.tmp
Удалён 4.tmp
Удалён 5.tmp
Готово. Удаление завершено.
tibezlepkina@fedora:~$
```

./files.sh 5 delete — файлы удалены. (рис. ??)

Рисунок 8 — листинг backup.sh. Скрипт переходит в каталог, find -mtime -7 ищет

```
GNU nano 8.5 backup.sh
#!/bin/bash

# Устанавливаем значения по умолчанию
DIR="${1:-.}" # если не передан аргумент, берём текущую директорию (..)
ARCHIVE="${2:-backup.tar}" # если не передан, имя архива = backup.tar

# Проверяем, существует ли указанная директория
if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория '$DIR' не существует"
    exit 1
fi

# Переходим в указанную директорию
cd "$DIR" || exit 1

# Проверяем, есть ли свежие файлы
echo "Поиск файлов, изменённых менее недели назад в директории: $(pwd)"

# Находим файлы и архивируем
```

файлы новее 7 дней, tar создаёт архив. (рис. ??)

Рисунок 8 — листинг backup.sh. Скрипт переходит в каталог, find -mtime -7 ищет

```
tibezlepkina@fedora:~$ chmod +x backup.sh
tibezlepkina@fedora:~$ ./backup.sh
Поиск файлов, изменённых менее недели назад в директории: /home/tibezlepkina
Архив успешно создан: /home/tibezlepkina/backup.tar
Содержимое архива:
./mozilla/extensions/{ec8030f7-c20a-464f-9b0e-13a3a9e97384}/.fedora-langpack-install
./mozilla/firefox/Crash Reports/crash_helper_server.log
./mozilla/firefox/Crash Reports/LastCrash
./mozilla/firefox/Crash Reports/submit.log
./mozilla/firefox/Crash Reports/pending/7e308349-0fc1-eee4-4383-2d0cbaf97b7d.dmp
./mozilla/firefox/Crash Reports/pending/7e308349-0fc1-eee4-4383-2d0cbaf97b7d.extra
./mozilla/firefox/Crash Reports/pending/3a3df0b9-de1d-4659-a5e4-3d40ec71d67a.dmp
./mozilla/firefox/Crash Reports/pending/3a3df0b9-de1d-4659-a5e4-3d40ec71d67a.extra
./mozilla/firefox/Crash Reports/pending/5aea50ec-31d3-3b4a-cf0b-b2140e5bb6bf.dmp
./mozilla/firefox/Crash Reports/pending/5aea50ec-31d3-3b4a-cf0b-b2140e5bb6bf.extra
./mozilla/firefox/Crash Reports/pending/55d7edad-348d-c84f-8127-b3310505fd30.dmp
./mozilla/firefox/Crash Reports/pending/55d7edad-348d-c84f-8127-b3310505fd30.extra
./mozilla/firefox/Crash Reports/pending/55462aea-2a44-93ea-949b-a4554ca59d18.dmp
./mozilla/firefox/Crash Reports/pending/55462aea-2a44-93ea-949b-a4554ca59d18.extra
./mozilla/firefox/Crash Reports/pending/3f57c7c0-e5ee-5ef9-90f7-7b1aa9733173.dmp
```

файлы новее 7 дней, tar создаёт архив. (рис. ??)

Выводы

В ходе выполнения лабораторной работы были изучены и практически применены основные конструкции программирования в командном процессоре UNIX: разбор опций с помощью `getopts`, условные операторы `if` и `case`, циклы `for` и `while`, работа с кодами возврата (`$?`), генерация имён файлов с метасимволами, а также утилиты `grep`, `tar`, `find`. Полученные навыки позволяют автоматизировать типовые задачи администрирования и обработки данных в среде UNIX. Использование команды `exit(n)` в программе на C/C++ продемонстрировало механизм взаимодействия между скомпилированной программой и оболочкой через код завершения.

Контрольные вопросы

1. Каково предназначение команды `getopts`? Команда `getopts` предназначена для разбора коротких ключей опций командной строки в `shell`-скриптах. Она позволяет последовательно обрабатывать переданные скрипту ключи, определять, требует ли каждый из них аргумента, и помещает этот аргумент в переменную `OPTARG`. `Getopts` автоматически учитывает стандартные соглашения UNIX, такие как объединение нескольких флагов в один, и после завершения разбора сдвигает указатель `OPTIND` на первый свободный аргумент. Это основной инструмент для создания гибких и удобных командных файлов с поддержкой опций.
2. Какое отношение метасимволы имеют к генерации имён файлов? Метасимволы, такие как звёздочка, вопросительный знак и квадратные скобки, используются оболочкой для так называемого глоббинга, то есть автоматической замены шаблона списком имён файлов, которые этому шаблону соответствуют. Например, шаблон звёздочка точка `txt` раскроется во все файлы с расширением `txt` в текущем каталоге, а вопросительный знак заменит ровно один любой символ. Это позволяет не перечислять имена файлов вручную, а задавать их кратко и гибко. Генерация имён файлов происходит до выполнения команды, поэтому сама команда получает уже готовый список имён.
3. Какие операторы управления действиями вы знаете? К основным операторам управления в командном процессоре UNIX относятся: `if` для условного выполнения, `case` для множественного выбора, `for`, `while`

и `until` для организации циклов, `break` и `continue` для прерывания цикла или перехода к следующей итерации, `return` для выхода из функции, `exit` для завершения скрипта с возвратом кода, а также логические операторы `амперсанд` `амперсанд` и двойная вертикальная черта, которые выполняют команды условно в зависимости от кода возврата предыдущей команды.

4. Какие операторы используются для организации цикла? Для организации циклов в `shell` используются три основных оператора: `for` для перебора заранее известного списка значений, например перебор всех файлов или чисел из диапазона; `while` для повторения блока команд, пока условие остаётся истинным, то есть код возврата равен нулю; `until` для повторения блока команд, пока условие является ложным, то есть код возврата не равен нулю. Также существует конструкция `select`, которая создаёт меню выбора, но она используется реже.
5. Для чего нужны команды `false` и `true`? Команда `true` всегда возвращает код завершения ноль, то есть успех, а команда `false` всегда возвращает ненулевой код, обычно единицу. Они нужны для создания бесконечных циклов, например `while true` с последующим `do` и `done`, для задания заведомо успешного или ошибочного исхода в условных конструкциях, а также для упрощения логики скриптов, когда требуется явно указать истину или ложь в месте, где ожидается выполнение команды.
6. Что означает строка, встреченная в командном файле? Эта строка проверяет, существует ли обычный файл с именем, которое формируется из значений переменных `s` и `i`. Конструкция `test -f` проверяет, является ли аргумент обычным файлом. Часть `mans/` означает подкаталог `man`, к которому добавляется содержимое переменной `s` и символ слеш. Далее идёт обратный слеш доллар `i` — это экранированный знак доллара с последующей буквой `i`, то есть в имени файла буквально присутствует символ доллар `i`, а не значение переменной. Затем следует обратный слеш

точка — экранированная точка, и в конце снова значение переменной `s`. Таким образом, полное имя файла выглядит как `map` значение `s` слеш доллар `i` точка значение `s`, и скрипт проверяет существование такого файла. Экранирование нужно, чтобы оболочка не интерпретировала специальные символы раньше времени.

7. Объясните различие между конструкциями `while` и `until`. Основное различие заключается в условии продолжения цикла. Конструкция `while` выполняет тело цикла до тех пор, пока условие истинно, то есть пока код возврата последней команды в условии равен нулю. Как только условие становится ложным, то есть код возврата не ноль, цикл прекращается. Конструкция `until`, напротив, выполняет тело цикла пока условие ложно, то есть пока код возврата не равен нулю. Цикл `until` продолжается до тех пор, пока условие не станет истинным, то есть код возврата не станет равным нулю. Проще говоря, `while` повторяет пока условие выполняется, а `until` повторяет пока условие не выполнится или до тех пор, пока условие не станет истинным. При инвертированном условии оба цикла могут дать одинаковый результат, но семантически они удобны в разных ситуациях: `while` удобен, когда нужно действовать, пока что-то верно, а `until` удобен, когда нужно ждать наступления некоторого события.

Список литературы

1. Mendel Cooper - Ссылка: <https://tldp.org/LDP/abs/html/>